Agentes Inteligentes

Selección de plataforma o framework para Agentes Inteligentes

Estudiante: Constanza Flores Vásquez
Profesor: Mauricio Araya
Fecha: Septiembre 2026

---

1. **¿Qué alternativas de implementación existen para el problema propuesto?**

   Para abordar el agente inteligente de Tetris, existen dos alternativas complementarias de implementación: un simulador lógico sin interfaz (*Headless Engine* en Python) y un videojuego interactivo desarrollado en Pygame. El simulador lógico es un motor ligero enfocado exclusivamente en la matemática de la partida, donde se representa el tablero como una matriz de 10×20 en memoria para evaluar rotaciones, colisiones y limpiezas de líneas en microsegundos, lo que permite ejecutar de forma prácticamente instantánea las partidas y así evaluar la función de aptitud (*fitness*) definida en la entrega anterior. Por otra parte, la interfaz gráfica implementada en Pygame proporciona el soporte de renderizado 2D, control de eventos y temporización de fotogramas, sustituyendo las pulsaciones de teclado del usuario por las acciones del agente, lo que permite observar, validar y demostrar en tiempo real el comportamiento y la estrategia optimizada por el sistema.

2. **¿Qué tan bien representa cada alternativa el problema original?**

   El simulador lógico representa de manera directa y fiel todas las mecánicas estructurales del juego: la cuadrícula, las formas de las piezas, las colisiones, el cálculo de huecos y la eliminación de líneas. Solo requiere adaptar la física de caída continua hacia un modelo de turnos discretos con caída instantánea (*hard drop*) y prescindir por completo del renderizado gráfico, lo cual elimina el retardo visual y hace computacionalmente factible evaluar miles de partidas acotadas a 1.000 piezas en poco tiempo. Por su parte, Pygame permite implementar de forma nativa la presentación del tablero, los colores, el marcador y la apreciación del juego en tiempo real, pero exige adaptar el bucle de control para deshabilitar las entradas del teclado del usuario y permitir que el agente inyecte sus decisiones de forma autónoma; no obstante, utilizar únicamente la interfaz gráfica para el entrenamiento masivo resulta inviable debido a la sobrecarga y lentitud que introduce la tasa de refresco (FPS).

3. **¿Fue posible instalar y ejecutar cada alternativa?**

   Fue posible instalar y ejecutar exitosamente ambas alternativas en el equipo de trabajo sin presentar conflictos de dependencias, utilizando un entorno de Python 3.11 gestionado mediante el administrador de paquetes `pip`. La lógica matricial del simulador se verificó ejecutando pruebas en consola sin interfaz gráfica, mientras que la biblioteca *Pygame* se

configuró de forma correcta, permitiendo inicializar la ventana y renderizar el tablero a 25 FPS estables, tal como se evidencia en la Figura 1.

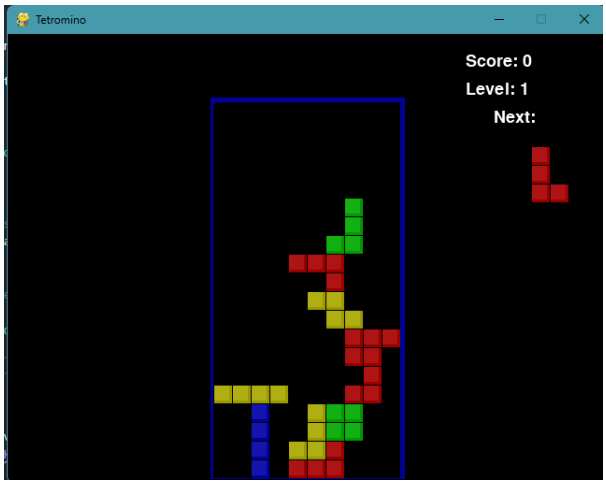


Figura 1: Ejecución exitosa de la interfaz gráfica desarrollada en Pygame.

4. **¿Es posible controlar un agente mediante código?**

Sí, es totalmente posible y se comprobó mediante la implementación de un agente reactivo mínimo (`SimpleAgent`) desarrollado en un módulo independiente (`test_agent.py`) e integrado en el bucle principal de `tetris.py`. En esta prueba mínima, el entorno expone el estado del juego mediante la matriz del tablero (`board`), las definiciones geométricas de los tetraminós y la pieza activa en turno; el agente lee esta información programáticamente y determina la orientación y columna de destino, delegando en el motor la ejecución de una caída instantánea (*hard drop*) por software. En el listing 1 se muestra la integración de la llamada al agente dentro del bucle de juego, y en el listing 2 se presenta el registro de terminal donde se constata el ciclo continuo de observación (alturas y piezas) y acción programática sin intervención de teclado.

```
if fallingPiece == None:
    fallingPiece = nextPiece
    nextPiece = getNewPiece()
    lastFallTime = time.time()

    if not isValidPosition(board, fallingPiece):
        return

    # Consulta programatica al agente
    agent = SimpleAgent()
    target_rot, target_x = agent.choose_action(
        board,
        fallingPiece,
        PIECES,
        isValidPosition,
        BOARDWIDTH
    )
```

```
18
19      # Aplicacion de la decision
20      fallingPiece['rotation'] = target_rot
21      fallingPiece['x'] = target_x
22
23      # Caida instantanea (hard drop)
24      for i in range(1, BOARDHEIGHT):
25          if not isValidPosition(board, fallingPiece, adjY=i):
26              break
27      fallingPiece['y'] += i - 1
```

Listing 1: Integración programática del agente en el bucle principal de `tetris.py`

```
Pieza entrante : 'S'
Alturas columnas: [0, 0, 0, 0, 2, 2, 0, 0, 0, 0]
Jugadas legales : 17 combinaciones evaluadas
>> Accion tomada: Rotacion 1 | Columna 6

Pieza entrante : 'L'
Alturas columnas: [0, 0, 0, 0, 2, 2, 0, 0, 3, 2]
Jugadas legales : 34 combinaciones evaluadas
>> Accion tomada: Rotacion 2 | Columna 4

Pieza entrante : 'S'
Alturas columnas: [0, 0, 0, 0, 2, 4, 4, 4, 3, 2]
Jugadas legales : 17 combinaciones evaluadas
>> Accion tomada: Rotacion 0 | Columna 5

Pieza entrante : 'Z'
Alturas columnas: [0, 0, 0, 0, 2, 4, 5, 6, 6, 2]
Jugadas legales : 17 combinaciones evaluadas
>> Accion tomada: Rotacion 1 | Columna 2

Pieza entrante : 'I'
Alturas columnas: [0, 0, 0, 3, 4, 4, 5, 6, 6, 2]
Jugadas legales : 17 combinaciones evaluadas
>> Accion tomada: Rotacion 1 | Columna 0

Pieza entrante : 'O'
Alturas columnas: [4, 4, 4, 4, 4, 4, 5, 6, 6, 2]
Jugadas legales : 9 combinaciones evaluadas
>> Accion tomada: Rotacion 0 | Columna 6

Pieza entrante : 'O'
Alturas columnas: [4, 4, 4, 4, 4, 4, 5, 8, 8, 2]
Jugadas legales : 9 combinaciones evaluadas
>> Accion tomada: Rotacion 0 | Columna 3
```

Listing 2: Registro de ejecución en terminal: ciclo de observación y decisión del agente

5. **¿Qué restricciones técnicas y económicas impone cada alternativa?**

   Económicamente no se presentan restricciones para ninguna de las dos alternativas, ya que ambas se sustentan en herramientas de código abierto bajo licencias de libre uso. Respecto

a los requisitos de software, se establece el uso específico de **Python 3.11**, dado que garantiza total compatibilidad y disponibilidad de binarios precompilados estables para *Pygame*, evitando los problemas de compilación en dependencias C/SDL que suelen presentarse en versiones más recientes del intérprete. En cuanto al hardware, el simulador lógico impone demandas mínimas: es multiplataforma, no requiere aceleración gráfica por GPU, opera con menos de 100 MB de memoria RAM y su almacenamiento es despreciable; su única exigencia radica en el tiempo de cómputo en CPU durante la ejecución masiva de las metaheurísticas. Por su parte, la interfaz en Pygame requiere controladores de pantalla compatibles y está sujeta a un reloj de tiempo real (25 FPS) necesario para la apreciación humana, lo que introduce un retraso que vuelve inviable el entrenamiento masivo en dicho entorno, limitando su rol técnico exclusivamente a la fase de demostración visual.

6. **¿Cuál es la alternativa seleccionada y qué riesgos presenta?**

   La alternativa seleccionada es una solución híbrida y desacoplada: el simulador lógico en memoria (*Headless Engine*) se utilizará como motor principal de cómputo para la optimización mediante metaheurísticas, mientras que la interfaz en Pygame se reservará exclusivamente como herramienta de visualización y demostración cualitativa. Al comparar ambas opciones, Pygame resulta inviable para el entrenamiento debido al retardo impuesto por su reloj de refresco (25 FPS), pero es indispensable para que el evaluador observe la toma de decisiones en tiempo real; por el contrario, el simulador lógico carece de representación gráfica, pero ofrece la velocidad de procesamiento en CPU necesaria para evaluar cientos de partidas en pocos segundos.

   Respecto a los riesgos del proyecto, se descartan barreras por costos de licencias, problemas de instalación o falta de documentación, dado que ambas herramientas son libres, maduras y se validaron exitosamente. El principal riesgo técnico radica en los **tiempos de simulación y la variabilidad estocástica de Tetris**: una secuencia aleatoria desfavorable puede penalizar a un conjunto de pesos heurísticos adecuado, por lo que medir la aptitud (*fitness*) de forma confiable requiere promediar múltiples partidas por configuración, incrementando la carga computacional si la lógica no está optimizada; otro riesgo latente es la convergencia hacia **óptimos locales** si el espacio de búsqueda de la función de evaluación heurística presenta mesetas o trampas locales, riesgo que se mitigará mediante el uso de algoritmos de optimización diseñados específicamente para escapar de mínimos locales (como metaheurísticas basadas en poblaciones, mecanismos de mutación adaptativa o técnicas de enfriamiento simulado), garantizando además que el simulador y la interfaz gráfica compartan con exactitud las mismas reglas de transición.